NoSQL - MongoDB Problem Statements
SET-8

Problem 1 – University Course Enrollment System

Problem Statement

A university maintains course enrollment records in a MongoDB collection named **enrollments**. Each document includes enrollment ID, student name, course name, semester, grade, attendance percentage, and enrollment status. The academic department needs to identify students with low attendance, update grades after re-evaluation, and remove withdrawn students from the system. Reports are generated using complex filtering conditions involving multiple operators.

Questions:

1. Insert multiple enrollment records into the **enrollments** collection.
2. Retrieve students whose attendance percentage is less than 75 **or** grade equals **"F"** using **$or**.
3. Display student name and course name fields only using projection.
4. Update grade to **"B"** for students whose grade equals **"C"** using **updateMany()**.
5. Delete enrollment records where enrollment status equals **"Withdrawn"**.

Problem 2 – Banking Transaction Monitoring System

Problem Statement

A bank maintains transaction records in a MongoDB collection named **transactions**. Each document includes transaction ID, account number, transaction type, transaction amount, transaction date, and transaction status. The fraud detection team frequently analyzes

© 2025 GUVI GEEK NETWORK PVT LTD. All rights reserved. No part of this document may be reproduced or transmitted in any form or by any means, electronic, mechanical, photocopying, recording, or otherwise, without prior permission from GUVI.

transactions above certain thresholds, updates transaction status after verification, and removes invalid transactions from the database. The system must support filtering using both comparison and logical operators simultaneously.

Questions:

1. Insert multiple transaction records into the **transactions** collection.
2. Retrieve transactions where transaction amount is greater than 50,000 using **$gt**.
3. Retrieve transactions where transaction amount is greater than 10,000 **and** transaction type equals **"Online"**.
4. Update transaction status to **"Verified"** for transactions where account number equals **1234567890**.
5. Delete transactions where transaction status equals **"Invalid"**.

Problem 3 – Hotel Reservation Management System

Problem Statement

A hotel chain maintains reservation data in a MongoDB collection named **reservations**. Each document includes reservation ID, guest name, room type, booking date, check-in date, check-out date, and payment status. The hotel management retrieves reservations based on booking dates, updates payment status after payment confirmation, and removes canceled reservations. Reports often involve filtering using multiple logical conditions.

Questions:

1. Insert multiple reservation records into the **reservations** collection.
2. Retrieve reservations where payment status equals **"Pending"** using **$eq**.

© 2025 GUVI GEEK NETWORK PVT LTD. All rights reserved. No part of this document may be reproduced or transmitted in any form or by any means, electronic, mechanical, photocopying, recording, or otherwise, without prior permission from GUVI.

3. Retrieve reservations where check-in date is greater than **2024-01-01** using **$gt**.

4. Update payment status to **"Completed"** for reservations where guest name equals **"Ravi"**.

5. Delete reservations where payment status equals **"Canceled"**.

Problem 4 – IT Asset Management in an Organization

Problem Statement

A large organization maintains IT asset information in a MongoDB collection named **assets**. Each document includes asset ID, asset name, asset type, purchase date, warranty period, asset value, and asset status. The IT department frequently retrieves assets based on value ranges, updates asset status after maintenance, and removes retired assets from the system. The organization also needs to filter assets based on multiple conditions involving comparison and logical operators.

Questions:

1. Insert multiple asset records into the **assets** collection.

2. Retrieve assets whose asset value is greater than 50,000 using **$gt**.

3. Retrieve assets whose asset value is less than 20,000 **or** asset status equals **"Under Maintenance"** using **$or**.

4. Update asset status to **"Active"** for assets whose warranty period equals **3 years** using **updateMany()**.

5. Delete assets where asset status equals **"Retired"** using **deleteMany()**.

Problem 5 – E-Commerce Order Tracking System

© 2025 GUVI GEEK NETWORK PVT LTD. All rights reserved. No part of this document may be reproduced or transmitted in any form or by any means, electronic, mechanical, photocopying, recording, or otherwise, without prior permission from GUVI.

Problem Statement

An e-commerce company stores order information in a MongoDB collection named **orders**. Each document includes order_id, customer_id, order_date, order_status, total_amount, and delivery_city. Customers frequently search orders based on order status and delivery city. As order volume increased significantly, the company experienced slower order tracking queries. The database administrator decided to implement indexing strategies to improve query efficiency.

Questions:

1. Create a **single index** on the **order_status** field.
2. Create a **compound index** on **order_status** and **delivery_city** fields.
3. Retrieve all orders where order_status equals “Shipped” and delivery_city equals “Chennai”.
4. Use **explain()** to analyze query execution statistics.
5. Compare the number of documents examined before and after indexing.

© 2025 GUVI GEEK NETWORK PVT LTD. All rights reserved. No part of this document may be reproduced or transmitted in any form or by any means, electronic, mechanical, photocopying, recording, or otherwise, without prior permission from GUVI.